

OOPS - Introduction

Example 1

```
a = 100
print(type(a))    # <class 'int'>
print(len(a))     # TypeError
```

Explanation

In the above example, finding the length of an integer is not possible because the int object does not have a len() method.

Example 2

```
l = [34, 23, 32, 42]
print(type(l))    # <class 'list'>
print(len(l))     # 4
```

Explanation

In the above example, finding the length of a list is possible because the list object has a len() method.

Internally, len() calls the special method (dunder method) `__len__()`.

`print(len(l))` is equivalent to `print(l.__len__())`

Syntax: dir(object)

`dir()` displays all the attributes available for an object.

Example 3

```
a = 10
b = 20
print(a + b)
```

Internally, `a + b` is equivalent to `a.__add__(b)`

Explanation

Since a and b are integer objects, Python internally calls the special method `__add__()`.

Example 4:

```
a = 100  
print(a.__len__())
```

Output

AttributeError: 'int' object has no attribute '__len__'

Explanation

An integer object does not contain the `__len__()` method.

Example 5

```
s1 = {23, 56, 89}  
s1.append(3)
```

Output

AttributeError: 'set' object has no attribute 'append'

Explanation

A set object does not contain an `append()` method.

Important Point:

We get an `AttributeError` whenever we try to access an attribute or method that does not exist in the object.

Attributes: Attributes are the members of a class.

- All states and behaviours are defined inside a class.
 - Every built-in datatype in Python has its own respective class.
 - Objects can access only the methods defined in their class.
-

OOPS (Object-Oriented Programming System)**Why OOPS?**

OOPS is required to create our own user-defined datatypes.

To create a new datatype, we need to create a class in which we define:

- States(Variables)
- Behaviours (Methods)

of an object.

Advantages of OOPS

- Helps model real-world entities.
 - Improves code reusability.
 - Improves maintainability.
 - Supports real-time scenarios.
 - Supports concepts like operator overloading and polymorphism.
-

Class

A class is

- A design
- A template
- A blueprint
- A plan

for creating objects.

Syntax to Create a Class in Python

```
class CLASSNAME:  
    # Attributes
```

Attributes

1. States(Variables)
 - Class variables
 - Instance variables
 2. Behaviours(Methods)
 - Class method
 - Static method
 - Instance method
 - Normal / User-defined Instance method
 - Dunder / Magic Instance method
-

Example: Creating an empty class

```
class Student:  
    pass
```

```
print(dir(Student))
```

-
- A class does not occupy memory until an object is created from it.
 - Even though Student is an empty class, Python automatically has 29 dunder methods.
 - These special methods are called **Dunder Methods (Magic Methods)** because their names begin and end with double underscores (`__`).
-

Object

An **object** is an **instance of a class**.

When a class is created, no memory is allocated. Memory is allocated only when an object is created from that class.

Syntax to Create an Object

```
object_name = ClassName( args )
```

Example:

```
class Student:  
    pass
```

```
s1 = Student()  
s2 = Student()  
s3 = Student()
```

Here:

- s1, s2, and s3 are different objects of the same class.
- One class can have any number of objects.

Instance:

When an object is created, memory is allocated, and that memory is called an instance.

An instance is the memory created or memory representation of a class.

Real time example:

```
class EMPLOYEE:

    company = "TCS"      # Class Variable

    def __init__(self, emp_name):
        self.emp_name = emp_name  # Instance Variable

    def work(self):      # Normal Instance Method
        print(self.emp_name, "is working")

# Object Creation
e1 = EMPLOYEE("Ravi")
```

Accessing Class Variables

```
Using class name
print(EMPLOYEE.company)  #TCS

Using object name
print(e1.company)  #TCS
```

Accessing Instance Variables

```
Using class name
print(EMPLOYEE.emp_name)
```

Output: AttributeError:type object ' EMPLOYEE ' has no attribute 'emp_name'

Because emp_name belongs to the object e1, not to the class EMPLOYEE.

```
Using object name
print(e1.emp_name) #Ravi
```

Calling Instance method

```
Using Object name
e1.work() #Ravi is working
```

Using Class name

EMPLOYEE.work() #TypeError

When calling an instance method using the class name, an object must be passed explicitly as an argument.

EMPLOYEE.work(e1) #Ravi is working

constructor(__init__)

- `__init__` is a special method in Python classes, called as constructor.
- It runs automatically when a new object of a class is created.
- It is mainly used to initialize instance variables.
- The first parameter is always `self`, which refers to the instance being created.

Note:

- class variables are one copy per class that is a common memory will be allocated.
- class variables are loaded to the memory during class loading time.
- Instance variables are one copy per object.
- instance variables are loaded to the memory during object creation time.

self:

- `self` is a parameter in Python is used inside methods which refers to the current instance of the class.
 - It is used to access the instance variables and instance methods of the current object.
 - It must be the first parameter in instance methods, including `__init__`.
 - When you call a method on an object, Python automatically passes the object as the first argument (`self`).
 - You can use any name instead of `self`, but not recommended.
-

Encapsulation

Encapsulation is the process of wrapping or binding data(variables and methods) into a single unit called a class.

Example:

class STUDENT:

```
# Class Variables
```

```
institute_name = "PySpiders"
```

```
location = "BTM"
```

```
def __init__(self, sid, name, qual, stream, yop, contact, address, gender):
```

```
    self.sid = sid
```

```
    self.name = name
```

```
    self.qual = qual
```

```
    self.stream = stream
```

```
    self.yop = yop
```

```
    self.contact = contact
```

```
    self.address = address
```

```
    self.gender = gender
```

```
# Instance Method
```

```
def display(self):
```

```
    print(f"Student ID : {self.sid}")
```

```
    print(f"Student Name : {self.name}")
```

```
    print(f"Qualification : {self.qual}")
```

```
    print(f"Stream : {self.stream}")
```

```
    print(f"Year of Passout : {self.yop}")
```

```
    print(f"Contact Number : {self.contact}")
```

```
    print(f"Address : {self.address}")
```

```
    print(f"Gender : {self.gender}")
```

```
    print(f"Institute Name : {Student.institute_name}")
```

```
    print(f"Location : {Student.location}")
```

```
# Object Creation
```

```
s1 = STUDENT( 'S1', "Charvi", "BE", "CSE", 2025, 9876543210, "Mysore", "F")
```

```
# Calling Instance Method
```

```
s1.display()
```